

Worksheet 2: test design techniques and coverage

Calidad y Automatización en Ingeniería de Software, Semestre 2 2026

Individual work. Worth a fraction of the **Hojas de Trabajo** rubric (2.5 pts of the course zona, split across every worksheet of the semester). Due **Tuesday 25 August, 23:59**, in the Hoja de Trabajo 2 section in MiU and nowhere else. The file is named `carnet.zip`, using the student's own `carné` number: for example, `20250123.zip`.

Objective: design a test suite for a Python project that already exists, applying the techniques from deck 04, and show with `pytest-cov` that coverage went up. What is graded is the choice of technique for each part of the code, and the argument behind it, more than the number of test cases.

1. The code

<https://github.com/danihrndzld/hdt2-bikeshare>

It is the fare and rental core of a bike-sharing service: time-based fares, surcharges, rider authorization, rental lifecycle and station occupancy. It runs:

```
git clone https://github.com/danihrndzld/hdt2-bikeshare.git
cd hdt2-bikeshare && uv sync && uv run bikeshare demo
```

The repository's `README.md` carries the **business rules**. That is the **test basis**, so test cases are designed against those rules and not against what the code does, which may well fail to comply with them.

It also ships a small suite in `tests/test_smoke.py` that walks the happy path only. That suite is left exactly as it is, and new test cases go in separate files inside `tests/`.

2. What to do

On the test level. Black-box test techniques are normally explained against the system as a whole (i.e. End 2 End/System Testing), but in this worksheet they are applied to **unit tests**, for practicality: the code is Python and it is tested with `pytest`, function by function. The same techniques come back later in the course on API integration testing, in practice with Bruno.

- Measure the before**, with the suite exactly as it ships, without adding a single test case yet:


```
uv run pytest --cov --cov-report=html:cobertura_antes --cov-report=term
```

 The coverage configuration already lives in `pyproject.toml`: it measures the package, includes branches and leaves the command line out.
- Decide which technique goes where.** Read the business rules and the code, and distribute the four black-box techniques from deck 04 across the parts of the system. Neither this handout nor the repository says which one belongs to which: that decision is the exercise, and it is argued in writing. Statement and branch testing are not part of the map, because the coverage report measures both directly.
- Write one to three test cases per technique that applies.** The first line of every test docstring, meaning the text between triple quotes right under the `def`, carries **only** one of these four words, spelled exactly like this:


```
equivalence boundary decision-table state-transition
```

```
def test_fare_at_the_close_of_the_first_hour():
    """boundary
    60 minutes is the last minute of the first hour."""
```

 It is the only fixed format required, because it ties every test case to its row in the traceability table. The rest of the docstring, the name of the test case and what it asserts are each student's call.
- Measure the after**, with the same command, into the `cobertura_despues` folder.
- Document** everything in the design document of section 4.

3. When a test case fails

A test case can fail because it is written wrong or because it found a defect. Telling those two apart is part of the work. If the test case is correct and the code does not comply with the test basis in `README.md`, the test case is neither deleted nor bent to fit the code: the defect is documented and the test case is left marked, with the reason written out.

```
@pytest.mark.xfail(reason="the README says X for this input, the code gives Y")
```

The repository already has `xfail_strict` turned on, so a marked test case that starts passing breaks the run and says so. The other valid option is to fix the code and explain the change. Either way the finding is recorded in the design document.

4. The design document

Submitted as `diseño-de-pruebas.md`, or as a PDF with that same name. It contains, in this order:

1. **The technique map.** A table with the four black-box techniques from deck 04: Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing and State Transition Testing. For each, three columns: whether it applies in this project, on which function or class, and **why that technique and not another**. A technique that does not apply is declared all the same, with the reason. This table carries the most weight of the submission.
2. **The work products** of each technique that does apply, in the format the deck used: the table of equivalence partitions with a representative value each, the boundary table with the exact value and its two neighbours, the full decision table with its 2^n columns, or the state model with its valid and invalid transitions.
3. **The defects**, if any turned up: which rule of `README.md` is not met, which input reveals it, in which file and line it lives, and what was done about it.
4. **The traceability table**, one row per test case written: name of the test case, technique, file and line of the code it exercises, and which defect it would catch.
5. **The coverage comparison**: total statements, covered before, covered after, percentage before, percentage after and the difference. Also branch coverage before and after, which is the one that reveals what statement coverage hides.

5. How much coverage has to go up

Statement coverage goes up by at least **5 percentage points** over the baseline measured in step 1. The numbers in the document have to match the HTML reports submitted.

6. What goes in the ZIP

- The complete repository with the new test cases inside it, including `pyproject.toml` and `uv.lock`.
- The `.git` folder with the full history, which survives compression. That is why the repository is cloned with `git clone` instead of downloaded as a ZIP from GitHub. Commit as the work progresses: a single commit with everything in it shows no process.
- `cobertura_antes/` and `cobertura_despues/`: both complete HTML reports, with their `index.html` and the files the tool generates. Screenshots do not count, and neither does a single-page HTML.
- `diseño-de-pruebas.md`, or the equivalent PDF.
- `resultado_pytest.txt`: the full output of the final run, copied exactly as it comes out of the terminal, unedited and untrimmed.

Without both HTML reports there is no before and after to compare, and without the `.git` folder there is no history to review. In either case the submission is not graded.

7. What is graded

1. **The reasoning.** The technique map puts each technique where it actually pays off, and the justification holds up. A technique applied where it adds nothing, or dismissed without an argument, costs the same as not having used it.
2. **The work products.** Every technique declared applicable brings its complete table or model. A decision table missing columns, or partitions that are really several values of the same range, count as a technique not applied.
3. **The defects.** Whichever ones turn up are documented against the rule of `README.md` they break, not against what the code does.
4. **The traceability.** Every test case declares its technique, appears in the table and points at code that exists. A row citing a line that does not match invalidates the row.
5. **The coverage.** The minimum in section 5 is met and the numbers square with the reports.
6. **The suite.** It runs clean with `uv sync` and `uv run pytest`, with no disabled or empty test cases, and none of them asserts something trivially true to inflate the number.

One warning about coverage, already made in deck 04: covering a line is not the same as checking it. A suite that reaches 100% without asserting anything useful does not pass this worksheet, however green the report looks.